

Ministry of Education and Science of Ukraine
National Technical University of Ukraine
"Kyiv Polytechnic Institute named after Igor Sikorsky"
Educational and Scientific Institute of Atomic and Thermal Energy
Department of Digital Technologies in Energy

Graphics work

Topic: «Operations on texture coordinates»
from the discipline: «Visualization of graphical and geometric information»

Variant 14

Performed by:
student of the TP-52mp group
Striltsov D.S.

Kyiv – 2025

1.Task

- Reuse texture mapping from Control task.
- Implement texture scaling (texture coordinates) scaling / rotation around user specified point- odd variants implement scaling, even variants implement rotation
 - It has to be possible to move the point along the surface (u,v) space using a keyboard. E.g. keys **A** and **D** move the point along u parameter and keys **W** and **S** move the point along v parameter.

2. Theoretical information

2.1 Parametric Surface Representation

In this work, the following analytical form of the surface is used:

$$x(u,v) = 2(u - 0,5),$$

$$y(u,v) = 2(v - 0,5),$$

$$z(u,v) = \frac{[2(u-0,5)]^3}{3} - \frac{[2(v-0,5)]^2}{2}$$

After replacing $X = 2(u - 0,5)$, $Y = 2(v - 0,5)$ we get the classical form:

$$z = \frac{X^3}{3} - \frac{Y^2}{2}$$

This surface belongs to the class of **saddle-shaped** (hyperbolic paraboloid with cubic nonlinearity along the X axis). It is characterized by:

- negative Gaussian curvature at most points,
- the presence of a saddle point in the center ($u = v = 0.5$),
- pronounced asymmetry along the u axis.

2.1.2 Calculating Normals Taking into Account Angle-Weighted Normals

For high-quality illumination and smoothness of the surface, normals are calculated using the **angle-weighted vertex normals** method:

1. For each face (triangle), the plane normal vector is calculated using the vector product:

$$\mathbf{N} = (\mathbf{P}_2 - \mathbf{P}_1) \times (\mathbf{P}_3 - \mathbf{P}_1)$$

2. For each vertex of the triangle, the angle at this vertex is determined (using the scalar product and arccosine).
3. The contribution of the face normal to the vertex normal is proportional to the angle at this vertex.
4. After accumulating all contributions for each vertex, normalization is performed:

$$\mathbf{n} = \frac{\sum (\mathbf{N}_i \cdot w_i)}{\| \sum (\mathbf{N}_i \cdot w_i) \|}, w_i = \text{angle at vertex}$$

This approach provides significantly better visual smoothness compared to simple averaging of normals of neighboring faces, especially on surfaces with different density of triangles and sharp corners.

2.3 Construction of tangent space and normal mapping

To implement **normal mapping**, it is necessary to construct a local coordinate system (TBN — Tangent, Bitangent, Normal) for each vertex.

Calculation algorithm:

1. For each face, the vectors $\Delta\mathbf{P}_1$, $\Delta\mathbf{P}_2$ and $\Delta\mathbf{UV}_1$, $\Delta\mathbf{UV}_2$ are calculated from the vertex coordinates and texture coordinates.
2. The system of equations is solved:

$$\mathbf{T}' = \mathbf{T} - \frac{\mathbf{T} \cdot \mathbf{N}}{\mathbf{N} \cdot \mathbf{N}} \mathbf{N}$$

The matrix of the system is inverted, which gives $\mathbf{T}$ and $\mathbf{B}$.

3. The tangent $\mathbf{T}$ is orthogonalized with respect to the normal $\mathbf{N}$ using the Gram-Schmidt method:

$$\mathbf{T}' = (\mathbf{T} * \hat{N})\hat{N}$$

4. The **handedness** (the sign of the scalar product $\mathbf{B}$ with $\mathbf{N} \times \mathbf{T}'$) is preserved, which is necessary for the correct reconstruction of the bitangent in the shader.
5. In the fragment shader, the TBN matrix is constructed:

$$\mathbf{M}_{\text{TBN}} = [\mathbf{t}' \quad \mathbf{b}' \quad \mathbf{n}]$$

The normal from the texture is transformed from the range $[0,1]$ to $[-1,1]$ and applied:

$$\mathbf{n}_{\text{world}} = \mathbf{M}_{\text{TBN}} \cdot (2 \cdot \text{texture}(u_{\text{normal}}, uv) - 1)$$

2.4 Transforming texture coordinates around an arbitrary center

To allow rotation and scaling of the texture around an arbitrary point (u_c, v_c) , the classical sequence of affine transformations is applied in the vertex shader:

$$\begin{aligned} \mathbf{uv}' &= (u, v) - (u_c, v_c) \\ \mathbf{uv}'' &= R(\theta) \cdot \mathbf{uv}' \\ \mathbf{uv}''' &= s \cdot \mathbf{uv}'' \\ \text{final } uv &= \mathbf{uv}''' + (u_c, v_c) \end{aligned}$$

where $R(\theta)$ is the rotation matrix by angle θ :

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

This approach allows the user to interactively manipulate the texture (scale, rotate, center shift) without overlapping edges and artifacts.

2.5 Lighting model

A modified **Phong** model with **Blinn-Phong** elements is used:

- **Ambient lighting** — constant contribution of the diffuse color texture

$$\text{ambient} = k_a \cdot \text{diffuseTex}$$

- **Diffuse lighting** (Lambert) — using the normal from the normal map:

$$\text{diffuse} = k_d \cdot \max(\mathbf{n} \cdot \mathbf{l}, 0) \cdot \text{diffuseTex}$$

- **Specular reflection** — using the Blinn-Phong model using an inverted roughness texture:

$$\text{specular} = k_s \cdot (\mathbf{n} \cdot \mathbf{h})^{\text{shininess}} \cdot (1 - \text{roughnessTex})$$

where **h** is a half-vector between the direction of the light source and the direction of the observer.

This combination allows you to obtain realistic lighting of a wooden surface with pronounced highlights and relief.

3. Implementation Details

3.1 Application Architecture

The application is implemented using pure **WebGL 1.0** and consists of the following modules:

Module	Description
Model.js	Generation of parametric geometry, normals, tangents, and buffers
main.js	WebGL initialization, input handling, uniform updates, and render loop
shader.gpu	Vertex and fragment shaders
index.html	User interface and control elements

3.2 Geometry Generation

The surface is discretized into a regular grid in parameter space. Each rectangular cell is split into two triangles. Vertex positions, texture coordinates, indices, normals, and tangents are generated dynamically based on the selected resolution.

When the resolution changes, buffers are rebuilt to maintain stability and performance.

3.3 Shader Programs

Vertex Shader:

- Performs coordinate transformations
- Applies texture coordinate transformations
- Passes tangent space data to the fragment shader

Fragment Shader:

- Reconstructs the TBN matrix
- Applies normal mapping
- Computes lighting using the modified Phong model

3.4 Interaction and Controls

User interaction includes:

- Mouse-based camera rotation
- Slider-based parameter adjustment
- Keyboard-based movement of the texture transformation center

3.5 Textures

Three high-resolution textures are used:

- **Diffuse map** – Base color information
- **Roughness map** – Surface material properties
- **Normal map** – Surface detail enhancement

Textures are loaded asynchronously and mipmaps are generated for improved rendering quality.

4. User's instruction

4.1 Interface Overview

4.1.1 Main Canvas Area

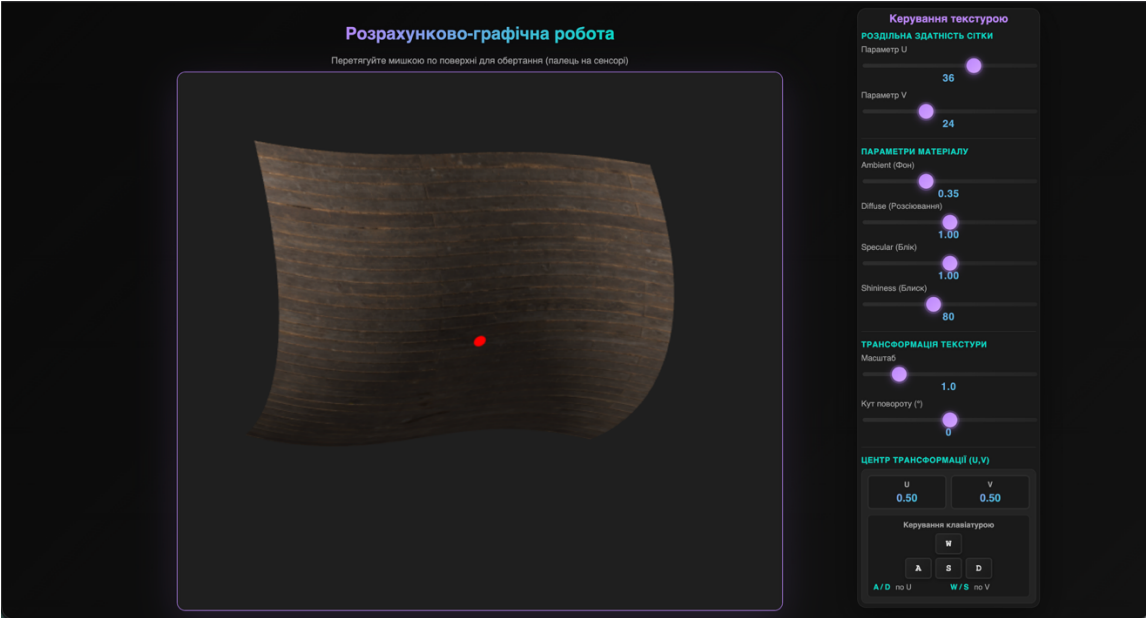


Figure 1 – Main Canvas

Figure 1 shows the main interface of the developed WebGL application. The central area displays the parametric surface with applied textures and lighting. The control panel on the right allows real-time adjustment of mesh resolution, texture transformation parameters, transformation center and change in lighting. Keyboard controls are used to shift the texture transformation center along the UV coordinates.

4.1.2 Control Panels

Action	Control
Camera rotation	Left mouse button + drag
Change U resolution	Slider "Parameter U"
Change V resolution	Slider "Parameter V"
Change Ambient	Ambient Intensity Slider
Change Diffuse	Diffuse Intensity Slider

Action	Control
Change Specular	Specular Intensity Slider
Change Shininess	Shininess Intensity Slider
Texture scale	Slider "Scale"
Texture rotation	Slider "Rotation Angle"
Move texture center (U)	Keys A (-) / D (+)
Move texture center (V)	Keys W (+) / S (-)

C. Texture Transformation Controls

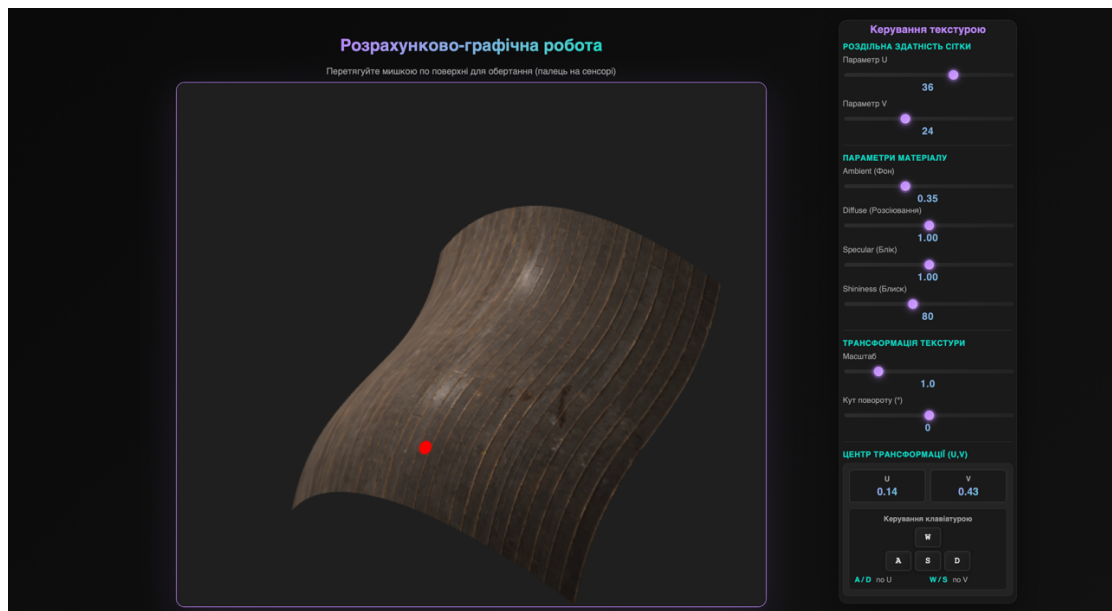


Figure 2 – Surface with applied texture controls

- Texture Scale U: Horizontal texture scaling (10-50)
- Texture Scale V: Vertical texture scaling (10-50)
- Texture Ambient: Ambient texture intensity (0.00–1.00)
- Texture Diffuse: Diffuse texture intensity (0.00–1.00)
- Texture Specular: Specular texture intensity (0.00–2.00)
- Texture Shininess: Shininess texture parameter (1–200)
- Texture Rotation: Rotate texture around center point (-180°; +180°)

- Point Size: Size of transformation point indicator (0.1-5.0)

D. Lighting Controls

```
gl.uniform3fv(shProgram.iAmbientColor, [ka, ka, ka]);  
gl.uniform3fv(shProgram.iDiffuseColor, [kd, kd, kd]);  
gl.uniform3fv(shProgram.iSpecularColor, [ks, ks, ks]);  
gl.uniform1f(shProgram.iShininess, shininess);
```

4.2 Step-by-Step Usage Guide

Step 1: Initial Setup

1. Open index.html in a compatible web browser
2. Wait for textures to load
3. The default view shows the Surface at 0° rotation

Step 2: Basic Navigation

1. Use the zoom and rotation sliders
2. Increase U/V Segments for smoother surface (may affect performance)
3. Adjust texture scale (U/V) to control texture density on the surface
4. Use the texture rotation slider to change texture orientation

Step 3: Texture Transformation

- Press W to move point upward in texture space
- Press S to move point downward in texture space
- Press A to move point left in texture space
- Press D to move point right in texture space

Note: The red point on the surface indicates the current transformation center.

5. Source code

```
'use strict';

function Model(name, gl, options) {
  this.name = name || 'ShoeSurface';
  this.gl = gl;
  options = options || {};

  this.uSteps = options.uSteps || 51;
  this.vSteps = options.vSteps || 51;

  const totalVertices = this.uSteps * this.vSteps;
  const totalIndices = (this.uSteps - 1) * (this.vSteps - 1) * 6;

  this.vertexBuffer = null;
  this.indexBuffer = null;
  this.normalBuffer = null;
  this.texcoordBuffer = null;
  this.tangentBuffer = null;

  this.vertices = new Float32Array(totalVertices * 3);
  this.normals = new Float32Array(totalVertices * 3);
  this.texcoords = new Float32Array(totalVertices * 2);
  this.tangents = new Float32Array(totalVertices * 4);
  this.indices = new Uint16Array(totalIndices);

  this.vertexCount = 0;
  this.indexCount = 0;

  this.surfaceFunc = options.surfaceFunc || Model.defaultSurface;

  this.vertexBuffer = gl.createBuffer();
  this.normalBuffer = gl.createBuffer();
  this.texcoordBuffer = gl.createBuffer();
  this.tangentBuffer = gl.createBuffer();
  this.indexBuffer = gl.createBuffer();
}

Model.defaultSurface = function(u, v) {
  var x = (u - 0.5) * 2.0;
  var y = (v - 0.5) * 2.0;
  var z = (x * x * x) / 3.0 - (y * y) / 2.0;
  return [x, y, z];
};

Model.prototype.resize = function(newUSteps, newVSteps) {
  this.uSteps = newUSteps;
  this.vSteps = newVSteps;

  const totalVertices = this.uSteps * this.vSteps;
```

```

const totalIndices = (this.uSteps - 1) * (this.vSteps - 1) * 6;

this.vertices = new Float32Array(totalVertices * 3);
this.normals = new Float32Array(totalVertices * 3);
this.texcoords = new Float32Array(totalVertices * 2);
this.tangents = new Float32Array(totalVertices * 4);
this.indices = new Uint16Array(totalIndices);

this.vertexCount = 0;
this.indexCount = 0;
};

Model.prototype.init = function() {
    var gl = this.gl;

    let iIndex = 0;

    const uStep = 1.0 / (this.uSteps - 1);
    const vStep = 1.0 / (this.vSteps - 1);

    for (let vi = 0; vi < this.vSteps; vi++) {
        const v = vi * vStep;
        for (let ui = 0; ui < this.uSteps; ui++) {
            const u = ui * uStep;
            const vertexIndex = vi * this.uSteps + ui;
            const p = this.surfaceFunc(u, v);

            const vOffset = vertexIndex * 3;
            this.vertices[vOffset] = p[0];
            this.vertices[vOffset + 1] = p[1];
            this.vertices[vOffset + 2] = p[2];

            this.normals[vOffset] = 0;
            this.normals[vOffset + 1] = 0;
            this.normals[vOffset + 2] = 0;

            const tcOffset = vertexIndex * 2;
            this.texcoords[tcOffset] = u;
            this.texcoords[tcOffset + 1] = v;
        }
    }

    for (let vi = 0; vi < this.vSteps - 1; vi++) {
        const rowOffset = vi * this.uSteps;
        for (let ui = 0; ui < this.uSteps - 1; ui++) {
            const p0 = rowOffset + ui;
            const p1 = p0 + 1;
            const p2 = p0 + this.uSteps;
            const p3 = p2 + 1;

            this.indices[iIndex] = p0;

```

```

        this.indices[iIndex + 1] = p1;
        this.indices[iIndex + 2] = p2;
        this.indices[iIndex + 3] = p2;
        this.indices[iIndex + 4] = p1;
        this.indices[iIndex + 5] = p3;

        iIndex += 6;
    }
}

this.calculateAngleWeightedNormals();

gl.bindBuffer(gl.ARRAY_BUFFER, this.vertexBuffer);
gl.bufferData(gl.ARRAY_BUFFER, this.vertices, gl.STATIC_DRAW);

gl.bindBuffer(gl.ARRAY_BUFFER, this.normalBuffer);
gl.bufferData(gl.ARRAY_BUFFER, this.normals, gl.STATIC_DRAW);

gl.bindBuffer(gl.ARRAY_BUFFER, this.texcoordBuffer);
gl.bufferData(gl.ARRAY_BUFFER, this.texcoords, gl.STATIC_DRAW);

gl.bindBuffer(gl.ARRAY_BUFFER, this.tangentBuffer);
gl.bufferData(gl.ARRAY_BUFFER, this.tangents, gl.STATIC_DRAW);

gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.indexBuffer);
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, this.indices, gl.STATIC_DRAW);

this.vertexCount = this.vertices.length / 3;
this.indexCount = this.indices.length;

gl.bindBuffer(gl.ARRAY_BUFFER, null);
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, null);
};

Model.prototype.calculateAngleWeightedNormals = function() {
    this.vertexCount = this.vertices.length / 3;
    this.normals.fill(0);
    let tangents = new Float32Array(this.vertexCount * 3);
    tangents.fill(0);
    let bitangents = new Float32Array(this.vertexCount * 3);
    bitangents.fill(0);

    const vec1 = new Float32Array(3);
    const vec2 = new Float32Array(3);
    const normal = new Float32Array(3);

    const vertices = this.vertices;
    const normals = this.normals;
    const indices = this.indices;

    const subtractVectors = (out, aoffset, boffset) => {
        out[0] = vertices[aoffset] - vertices[boffset];
        out[1] = vertices[aoffset + 1] - vertices[boffset + 1];
    };

```

```

    out[2] = vertices[aOffset + 2] - vertices[bOffset + 2];
};

const calculateTriangleNormal = (out, v1offset, v2offset, v3offset) => {
    subtractVectors(vec1, v2offset, v1offset);
    subtractVectors(vec2, v3offset, v1offset);

    out[0] = vec1[1] * vec2[2] - vec1[2] * vec2[1];
    out[1] = vec1[2] * vec2[0] - vec1[0] * vec2[2];
    out[2] = vec1[0] * vec2[1] - vec1[1] * vec2[0];
};

const calculateAngle = (v1offset, v2offset, v3offset) => {
    subtractVectors(vec1, v2offset, v1offset);
    subtractVectors(vec2, v3offset, v1offset);

    const len1Sq = vec1[0] * vec1[0] + vec1[1] * vec1[1] + vec1[2] * vec1[2];
    const len2Sq = vec2[0] * vec2[0] + vec2[1] * vec2[1] + vec2[2] * vec2[2];
    const dot = vec1[0] * vec2[0] + vec1[1] * vec2[1] + vec1[2] * vec2[2];

    const cos = dot / Math.sqrt(len1Sq * len2Sq);
    return Math.acos(Math.min(1.0, Math.max(-1.0, cos)));
};

for(let i = 0; i < indices.length; i += 3) {
    const idx1 = indices[i] * 3;
    const idx2 = indices[i+1] * 3;
    const idx3 = indices[i+2] * 3;

    calculateTriangleNormal(normal, idx1, idx2, idx3);

    const angle1 = calculateAngle(idx1, idx2, idx3);
    const angle2 = calculateAngle(idx2, idx3, idx1);
    const angle3 = calculateAngle(idx3, idx1, idx2);

    subtractVectors(vec1, idx2, idx1);
    subtractVectors(vec2, idx3, idx1);

    const vertexIndex1 = indices[i];
    const vertexIndex2 = indices[i+1];
    const vertexIndex3 = indices[i+2];

    const uvOffset1 = vertexIndex1 * 2;
    const uvOffset2 = vertexIndex2 * 2;
    const uvOffset3 = vertexIndex3 * 2;

    const delta_u1 = this.texcoords[uvOffset2] - this.texcoords[uvOffset1];
    const delta_v1 = this.texcoords[uvOffset2 + 1] - this.texcoords[uvOffset1 +
1];

    const delta_u2 = this.texcoords[uvOffset3] - this.texcoords[uvOffset1];
    const delta_v2 = this.texcoords[uvOffset3 + 1] - this.texcoords[uvOffset1 +
1];

```

```

const denom = delta_u1 * delta_v2 - delta_u2 * delta_v1;
if (Math.abs(denom) < 1e-6) continue;
const invDenom = 1 / denom;

const tan = new Float32Array(3);
const bi = new Float32Array(3);
for (let k = 0; k < 3; k++) {
    tan[k] = invDenom * (delta_v2 * vec1[k] - delta_v1 * vec2[k]);
    bi[k] = invDenom * (delta_u1 * vec2[k] - delta_u2 * vec1[k]);
}

for(let j = 0; j < 3; j++) {
    normals[idx1 + j] += normal[j] * angle1;
    tangents[idx1 + j] += tan[j] * angle1;
    bitangents[idx1 + j] += bi[j] * angle1;

    normals[idx2 + j] += normal[j] * angle2;
    tangents[idx2 + j] += tan[j] * angle2;
    bitangents[idx2 + j] += bi[j] * angle2;

    normals[idx3 + j] += normal[j] * angle3;
    tangents[idx3 + j] += tan[j] * angle3;
    bitangents[idx3 + j] += bi[j] * angle3;
}
}

const EPSILON = 1e-6;
for(let i = 0; i < normals.length; i += 3) {
    const x = normals[i];
    const y = normals[i+1];
    const z = normals[i+2];
    const lenSq = x*x + y*y + z*z;

    if(lenSq > EPSILON) {
        const invLen = 1 / Math.sqrt(lenSq);
        normals[i] *= invLen;
        normals[i+1] *= invLen;
        normals[i+2] *= invLen;
    }
}

const temp = new Float32Array(3);
const cros = new Float32Array(3);
const bitan = new Float32Array(3);
for (let vi = 0; vi < this.vertexCount; vi++) {
    const offset = vi * 3;
    const n = [normals[offset], normals[offset + 1], normals[offset + 2]];
    let t_acc = [tangents[offset], tangents[offset + 1], tangents[offset + 2]];

    let dot_tn = n[0] * t_acc[0] + n[1] * t_acc[1] + n[2] * t_acc[2];
    temp[0] = n[0] * dot_tn;
    temp[1] = n[1] * dot_tn;
    temp[2] = n[2] * dot_tn;
}

```

```

        t_acc[0] -= temp[0];
        t_acc[1] -= temp[1];
        t_acc[2] -= temp[2];

        let len_t = Math.sqrt(t_acc[0] * t_acc[0] + t_acc[1] * t_acc[1] + t_acc[2] *
t_acc[2]);
        if (len_t > EPSILON) {
            t_acc[0] /= len_t;
            t_acc[1] /= len_t;
            t_acc[2] /= len_t;
        } else {
            t_acc = [1, 0, 0];
        }

        cros[0] = n[1] * t_acc[2] - n[2] * t_acc[1];
        cros[1] = n[2] * t_acc[0] - n[0] * t_acc[2];
        cros[2] = n[0] * t_acc[1] - n[1] * t_acc[0];

        bitan[0] = bitangents[offset];
        bitan[1] = bitangents[offset + 1];
        bitan[2] = bitangents[offset + 2];
        let len_b = Math.sqrt(bitan[0] * bitan[0] + bitan[1] * bitan[1] + bitan[2] *
bitan[2]);
        if (len_b > EPSILON) {
            bitan[0] /= len_b;
            bitan[1] /= len_b;
            bitan[2] /= len_b;
        } else {
            bitan = [0, 1, 0];
        }

        let dot_b = cros[0] * bitan[0] + cros[1] * bitan[1] + cros[2] * bitan[2];
        let handedness = (dot_b >= 0) ? 1.0 : -1.0;

        const tanOffset = vi * 4;
        this.tangents[tanOffset] = t_acc[0];
        this.tangents[tanOffset + 1] = t_acc[1];
        this.tangents[tanOffset + 2] = t_acc[2];
        this.tangents[tanOffset + 3] = handedness;
    }
};

Model.prototype.Draw = function() {
    const gl = this.gl;
    if (!gl || !shProgram || !shProgram.prog ||
        shProgram.iAttribVertex === -1 || shProgram.iAttribNormal === -1 ||
        shProgram.iAttribTexcoord === -1 || shProgram.iAttribTangent === -1) {
        if (!gl) console.error('Draw: gl is null');
        if (!shProgram) console.error('Draw: shProgram is null');
        if (!shProgram || !shProgram.prog) console.error('Draw: shProgram.prog is
null');
        if (shProgram && (shProgram.iAttribVertex === -1 || shProgram.iAttribNormal
=== -1 ||

```



```

        shProgram.iAttribTexcoord === -1 || shProgram.iAttribTangent === -1)) {
        console.error('Draw: Missing attributes', {
            vertex: shProgram.iAttribVertex,
            normal: shProgram.iAttribNormal,
            texcoord: shProgram.iAttribTexcoord,
            tangent: shProgram.iAttribTangent
        });
    }
    return;
}

if (this.indexCount === 0) {
    console.error('Draw: indexCount is 0!');
    return;
}

const vertexLoc = shProgram.iAttribVertex;
const normalLoc = shProgram.iAttribNormal;
const texcoordLoc = shProgram.iAttribTexcoord;
const tangentLoc = shProgram.iAttribTangent;

gl.useProgram(shProgram.prog);

gl.enableVertexAttribArray(vertexLoc);
gl.enableVertexAttribArray(normalLoc);
gl.enableVertexAttribArray(texcoordLoc);
gl.enableVertexAttribArray(tangentLoc);

gl.bindBuffer(gl.ARRAY_BUFFER, this.vertexBuffer);
gl.vertexAttribPointer(vertexLoc, 3, gl.FLOAT, false, 0, 0);

gl.bindBuffer(gl.ARRAY_BUFFER, this.normalBuffer);
gl.vertexAttribPointer(normalLoc, 3, gl.FLOAT, false, 0, 0);

gl.bindBuffer(gl.ARRAY_BUFFER, this.texcoordBuffer);
gl.vertexAttribPointer(texcoordLoc, 2, gl.FLOAT, false, 0, 0);

gl.bindBuffer(gl.ARRAY_BUFFER, this.tangentBuffer);
gl.vertexAttribPointer(tangentLoc, 4, gl.FLOAT, false, 0, 0);

gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.indexBuffer);

const drawError = gl.getError();
if (drawError !== gl.NO_ERROR) {
    console.error('WebGL error before drawElements:', drawError);
}

gl.drawElements(gl.TRIANGLES, this.indexCount, gl.UNSIGNED_SHORT, 0);

const drawErrorAfter = gl.getError();
if (drawErrorAfter !== gl.NO_ERROR) {

```

```

        console.error('WebGL error after drawElements:', drawErrorAfter,
        'indexCount:', this.indexCount);
    }

    gl.disableVertexAttribArray(vertexLoc);
    gl.disableVertexAttribArray(normalLoc);
    gl.disableVertexAttribArray(texcoordLoc);
    gl.disableVertexAttribArray(tangentLoc);
};

Model.prototype.dispose = function() {
    var gl = this.gl;
    if (this.vertexBuffer) gl.deleteBuffer(this.vertexBuffer);
    if (this.normalBuffer) gl.deleteBuffer(this.normalBuffer);
    if (this.texcoordBuffer) gl.deleteBuffer(this.texcoordBuffer);
    if (this.tangentBuffer) gl.deleteBuffer(this.tangentBuffer);
    if (this.indexBuffer) gl.deleteBuffer(this.indexBuffer);

    this.vertexBuffer = null;
    this.normalBuffer = null;
    this.texcoordBuffer = null;
    this.tangentBuffer = null;
    this.indexBuffer = null;
    this.vertices = [];
    this.normals = [];
    this.texcoords = [];
    this.tangents = [];
    this.indices = [];
    this.vertexCount = 0;
    this.indexCount = 0;
};

Model.prototype.getCurveCount = function() {
    return {
        uCount: this.uSteps,
        vCount: this.vSteps,
        totalVertices: this.vertexCount
    };
};

Model.prototype.getMemorySize = function() {
    var vertexSize = this.vertices.length * 4;
    var normalSize = this.normals.length * 4;
    var texcoordSize = this.texcoords.length * 4;
    var tangentSize = this.tangents.length * 4;
    var indexSize = this.indices.length * 2;
    return (vertexSize + normalSize + texcoordSize + tangentSize + indexSize) /
    1024.0;
};

```